ASSIGNMENT No: 10

Name: Arhaan Hussain Khan

PRN: 25070122045

TITLE: Write a Java program to demonstrate the use of abstract classes and abstract methods.

Problem Statement: Write a Java program to demonstrate the use of abstract classes and abstract methods.

Learning Objectives: To implement abstraction using abstract classes and abstract methods.

Theory:

Abstract classes provide partial implementation of a program and cannot be instantiated directly. They may contain both abstract and concrete methods. Abstract methods declare functionality that must be implemented by subclasses. Abstraction hides implementation details while exposing only essential functionality. This improves code flexibility and maintainability in object-oriented programming.

Exercise

1.Create an abstract Payment class and implement Credit Card and UPI payment classes.?

PaymentDemo.java > PaymentDemo > main(String[])

```java
abstract class Payment {
    abstract void makePayment(double amount);
}
class CreditCardPayment extends Payment {
    void makePayment(double amount) {
        System.out.println("Paid Rs." + amount + " using Credit Card");
    }
}
class UPIPayment extends Payment {
    void makePayment(double amount) {
        System.out.println("Paid Rs." + amount + " using UPI");
    }
}
public class PaymentDemo {
    Run | Debug
    public static void main(String[] args) {

        Payment p1 = new CreditCardPayment();
        Payment p2 = new UPIPayment();

        p1.makePayment(amount: 5000);
        p2.makePayment(amount: 1500);
    }
}

```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\arhaa\OneDrive\Desktop\java>  & 'C:\Program Files\Java\jdk-26.0.1\bi
ng\Code\User\workspaceStorage\02e7f2cfce92a0ba8e30606ade6b2c24\redhat.java\jdt_w
Paid Rs.5000.0 using Credit Card
Paid Rs.1500.0 using UPI
PS C:\Users\arhaa\OneDrive\Desktop\java>
```

2. Create an abstract FoodOrder class with an abstract method calculateBill(). Implement two subclasses, DineInOrder and TakeAwayOrder, to calculate and display the total bill based on the order type.

FoodOrderDemo.java > FoodOrderDemo

```java
abstract class FoodOrder {

    abstract void calculateBill();
}

class DineInOrder extends FoodOrder {

    double amount = 500;

    void calculateBill() {
        System.out.println("Dine-In Bill: Rs." + amount);
    }
}

class TakeAwayOrder extends FoodOrder {

    double amount = 500;
    double packingCharge = 20;

    void calculateBill() {
        System.out.println("Takeaway Bill: Rs." + (amount + packingCharge));
    }
}
public class FoodOrderDemo {
    Run | Debug
    public static void main(String[] args) {

        FoodOrder f1 = new DineInOrder();
        FoodOrder f2 = new TakeAwayOrder();

        f1.calculateBill();
        f2.calculateBill();
    }
}
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\arhaa\OneDrive\Desktop\java>  & 'C:\Program Files\Java\jdk-26.0.1\bin\
ng\Code\User\workspaceStorage\02e7f2cfce92a0ba8e30606ade6b2c24\redhat.java\jdt_ws\
Paid Rs.5000.0 using Credit Card
Paid Rs.1500.0 using UPI
PS C:\Users\arhaa\OneDrive\Desktop\java>
```